M.Sriram (192311120)

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I **M.Sriram / 192311120** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***M.Sriram***

M.Sriram (192311120)

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - ANSWER SHEET

Course Name: Internet Programming | Course Code: CSA43

CO-1 (L2) | Assessment Tool: Data Interpretation (Medium) | Total Marks: 20

Name	M.Sriram	Register No.	192311120

Q1. HTTP Response Analysis

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Incorrect or invalid fields

- **Content-Length: -150** - invalid. Per HTTP, Content-Length must be a non-negative integer (number of body bytes); a negative size is meaningless.
- **Date: Mon, 01 Apr 2026** - the format is correct, but 1 April 2026 is a **Wednesday**, not Monday, so the day name does not match the date.
- **Content-Type: text/html** - valid, but best practice is to declare the charset (text/html; charset=UTF-8).
- **Connection: keep-alive** - valid (default in HTTP/1.1), but it depends on a correct Content-Length to work.

2. Issue with Content-Length and its impact

Content-Length tells the client exactly where the message body ends. With -150 the client cannot find the end of the body. Impact:

- Browsers/proxies treat the response as malformed, reject it, or close the connection; the page may fail to load or load truncated.
- With keep-alive, the next response on the same TCP connection cannot be separated correctly (message desynchronisation).
- Inconsistent length handling between proxy and server can enable HTTP request/response smuggling attacks.
- Caching and progress indicators break because the size is unknown.

3. Does the status code match the response correctness?

No. 200 OK says the request succeeded and the message is well-formed, but the header is invalid and the body cannot be framed. A server that generates an invalid length has an internal fault, so a **500 Internal Server Error** is more appropriate (or the header must be fixed so that 200 is truthful).

4. Corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150        <- must equal the real body size in bytes
Connection: keep-alive
```

Alternative: remove Content-Length and use `Transfer-Encoding: chunked` when the size is not known in advance.

Conclusion: The response is not standards-compliant because of the negative Content-Length (critical) and the wrong weekday (minor). Fixing both, and keeping the status consistent with the real outcome, gives a reliable and secure HTTP message.

Q2. DOM Structure Analysis

```
<html>
 <head>
  <title>TestPage</title>
 </head>
 <body>
  <div>
   <p>HelloWorld
  </div>
 </body>
</html>
```

1. Missing or improperly closed elements

- ** ** is not closed - the closing tag is missing before .
- No **<!DOCTYPE html>** declaration at the top.
- Recommended but absent: `lang` attribute on <html> and <meta charset="UTF-8"> in <head>.
- html, head, title, body and div are all properly opened and closed.

2. How the browser interprets this

The HTML5 parser does not stop on errors; it applies error recovery. When it meets while a is open, it automatically closes the paragraph (an *implied end tag*) and then closes the div. The resulting DOM is: `html > body > div > p > "HelloWorld"`. Without a DOCTYPE the browser enters **quirks mode**.

3. Impact on rendering and layout

- The text still appears, because the parser repairs the tree - but the source is invalid and relies on browser guessing.
- The paragraph keeps its default block styling (1em top and bottom margin) inside the div.
- Quirks mode can change the box model, table/line-height behaviour and font sizing, so the layout may differ between browsers.
- Without charset/lang, special characters and screen-reader pronunciation may be affected. Also, "HelloWorld" has no space, so it shows as one word.

4. Corrected, valid DOM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

Conclusion: The only structural error is the unclosed ; the browser hides it, but explicit closing tags and a DOCTYPE give predictable, standards-mode rendering.

Q3. GET vs POST in Login System

1. Differences in data transmission

Feature	GET	POST
Where data goes	Appended to the URL as query string (/login?user=admin&pass=1234)	Sent inside the HTTP request body
Visibility	Visible in address bar, history, bookmarks, server/proxy logs, Referer header	Not shown in URL; not stored in history or bookmarks

Feature	GET	POST
Caching	Can be cached and bookmarked	Not cached by default
Size limit	Limited by URL length	Practically no limit
Intended use	Safe, read-only retrieval	Submitting data that changes state / sensitive data
Security level	Low	Moderate (needs HTTPS to be truly safe)

2. Which method suits login, and why

POST is the suitable method. Credentials stay out of the URL, are not saved in browser history or bookmarks, are not recorded in access logs as part of the URL, and a login is a state-changing action (it creates a session), which matches POST semantics.

3. Security risks of GET for authentication

- Password exposed in the URL - visible to anyone looking at the screen (shoulder surfing).
- Stored in browser history, bookmarks and autocomplete.
- Recorded in web-server, proxy and CDN logs in plain text.
- Leaked to third-party sites through the Referer header.
- Can be cached, shared by link, and replayed; easier to attack with CSRF because a simple link can trigger login.

4. Best practices for secure login

- Use **POST over HTTPS/TLS** only; redirect HTTP to HTTPS and enable HSTS.
- Store passwords as salted hashes (bcrypt / Argon2), never in plain text.
- Protect against CSRF with tokens and set cookies as `HttpOnly; Secure; SameSite`.
- Limit failed attempts (rate-limiting, lockout, CAPTCHA) and add multi-factor authentication.
- Validate and sanitise input (prevent SQL injection/XSS); show generic errors ("invalid username or password").
- Never log passwords; regenerate the session ID after login and expire sessions.

Conclusion: GET is unsuitable for credentials; POST with HTTPS plus hashing, CSRF protection and rate-limiting is the secure approach.

Q4. HTML Code Inspection

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Missing closing tags

- **</h1>** is missing after "Welcome".
- **</p>** is missing after "This is a paragraph".
- <img> needs no closing tag in HTML5 (void element). In XHTML it must be self-closed: .

2. Issues with the <img> tag

- Missing **alt** attribute - required for valid HTML and accessibility (screen readers, shown if the image fails to load, SEO).
- No **width**/**height** - the browser cannot reserve space, causing layout shift while loading.

- Path `image.jpg` is relative; if the file is missing the image will be broken.
- Not self-closed, which is acceptable in HTML5 but invalid in XHTML/XML.

3. How browsers handle these errors

The HTML5 parser recovers silently. A <p> start tag does **not** close an open <h1>, so the paragraph (and the image) become nested *inside* the heading and everything is closed only at </body>. Visually, the paragraph text appears in large bold h1 style, and the whole block behaves as a heading (affecting document outline and screen readers). The image renders inline, with a broken-image icon if it is not found.

4. Corrected code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

Conclusion: Unclosed h1/p tags change the document structure, not just its look; adding closing tags and a proper alt attribute makes the page valid, accessible and predictable.

Q5. Browser-Server Communication Flow

1. Interpretation of each step

Step	What happens
1. User enters URL	Browser parses the URL into scheme (http/https), host, port, path and query.
2. DNS lookup	Domain name is resolved to an IP address (browser cache, OS cache, recursive resolver, root, TLD, authoritative server).
3. HTTP request sent	A TCP connection (and TLS handshake for HTTPS) is opened; browser sends e.g. GET /index.html with headers (Host, User-Agent, Accept, Cookies).
4. Server processes request	Web server routes the request, runs application logic, queries the database or reads files, builds the response.
5. Response returned	Server sends status line, headers (Content-Type, Content-Length, Cache-Control) and body (HTML).
6. Browser renders page	Browser parses HTML into the DOM, CSS into the CSSOM, runs JavaScript, fetches images/CSS/JS, then lays out and paints the page.

2. Where delays can occur

- **DNS lookup** - slow or uncached resolvers.
- **TCP/TLS handshake** - extra round trips, long distance to the server.
- **Server processing** - heavy code, slow database queries, overloaded server.
- **Network transfer** - large responses, low bandwidth, packet loss.
- **Rendering** - render-blocking CSS/JS, many sub-resource requests, big images.

3. Role of DNS

Computers communicate using IP addresses, while users use names. DNS is the "phone book of the Internet": it translates a domain name (e.g. www.example.com) to an IP address so the browser knows which server to contact. Without a successful DNS lookup, the HTTP request cannot even be sent. Results are cached (browser, OS, resolver) according to the record TTL to avoid repeating the full lookup.

4. Optimizations to improve performance

- Use DNS caching, a fast resolver and `dns-prefetch` / `preconnect` hints.
- Serve content from a **CDN** close to the user; use HTTP/2 or HTTP/3 and keep-alive connections.
- Enable GZIP/Brotli compression; minify HTML, CSS and JS; optimise and lazy-load images.
- Set caching headers (Cache-Control, ETag) and use server-side caching and database indexing.
- Avoid render-blocking resources (async/defer scripts, inline critical CSS), reduce number of requests.

Conclusion: Page-load time is the sum of DNS, connection, server, transfer and rendering time; measuring each stage and optimising the slowest gives the largest improvement.

Rubric Alignment (Self-Assessment Summary)

Criteria (Weightage)	How this submission addresses it
Understanding of Data (25%)	Every header, DOM, table, code snippet and flow step is interpreted exactly (e.g. negative Content-Length, wrong weekday, unclosed p, missing alt).
Analysis (25%)	Each error is explained with its cause, effect and browser/server behaviour (parser error recovery, desynchronisation, nesting inside h1).
Application of Concepts (20%)	HTTP, DOM, HTML5 parsing, GET/POST, HTTPS, DNS and performance concepts are applied with corrected code and best practices.
Conclusion (20%)	A clear conclusion is given at the end of every question.
Clarity (10%)	Numbered answers matching question order, tables, code blocks and correct terminology.

Code of Conduct

I **M.Sriram / 192311120** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***M.Sriram***